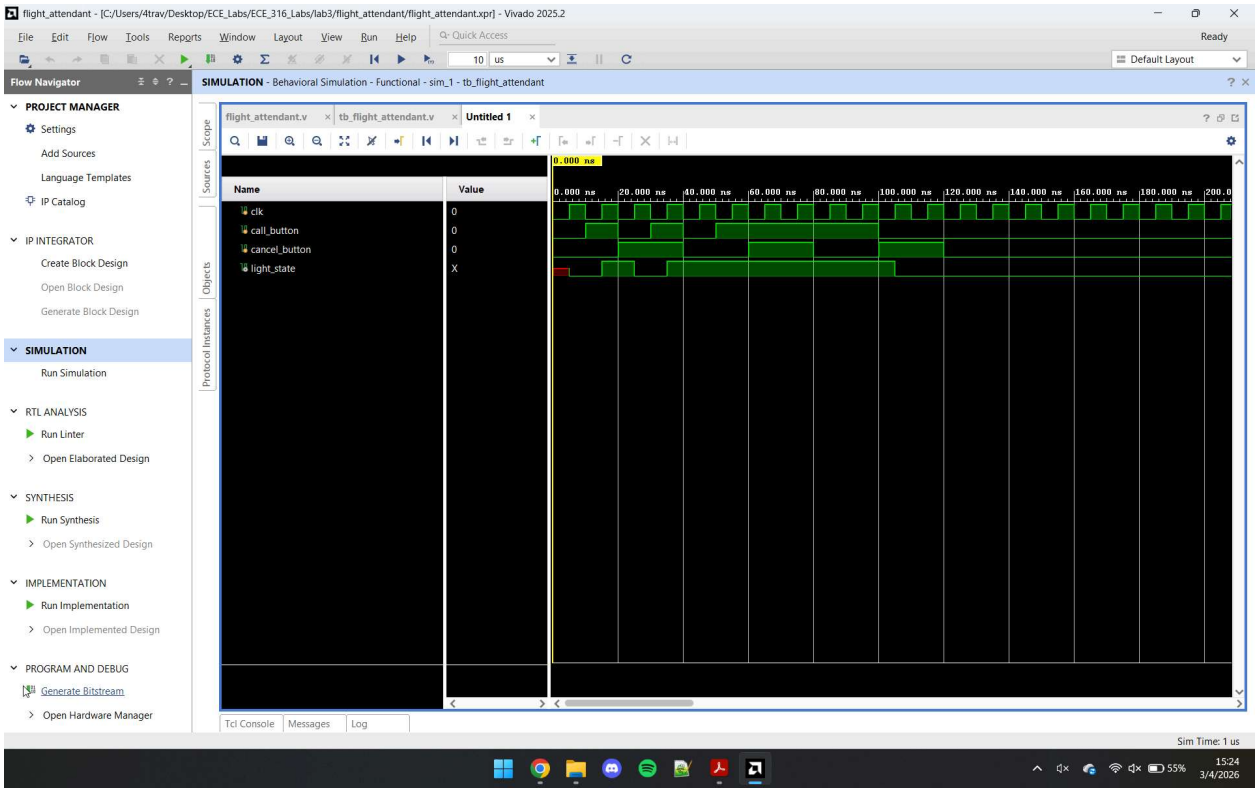
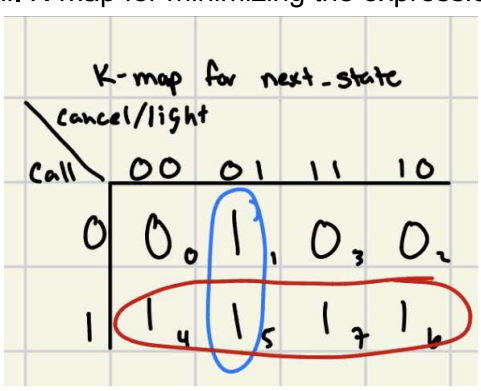
Part 1

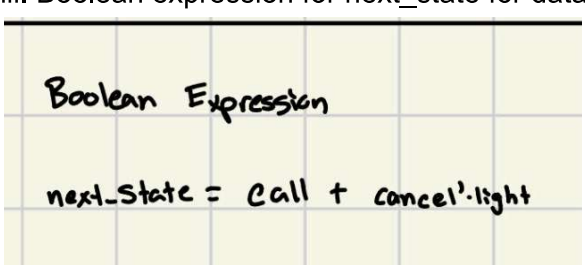
i. Simulation waveform of the Flight Attendant Call System for behavioral as well as dataflow modelling



ii. K-map for minimizing the expression for next_state for dataflow modelling



iii. Boolean expression for next_state for dataflow modelling

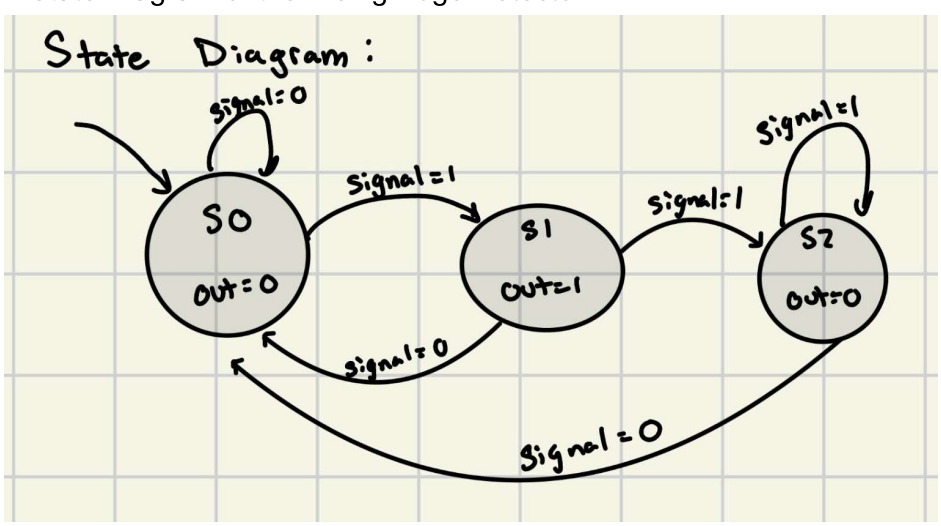


iv. Completed design file (.v) for dataflow modelling

```
module flight_attendant_dataflow(
input wire clk,
input wire call_button,
input wire cancel_button,
output reg light_state
);

wire next_state;

// Combinational Part of the Code
assign next_state = (call_button) + (light_state & (~cancel_button)) ;

// Sequential part of the code
always @(posedge clk) begin
   light_state <= next_state;
end

endmodule
```

Part 2

v. State Diagram of the Rising Edge Detector



Note - S0 is initial/wait for high, S1 is edge detected/output a high, S2 is wait for low

vi. Completed design files (.v) including the top module and clock divider

```
module clkdiv(
   input clk,
   input reset,
```

```
    output clk_out
);

reg [26:0] COUNT; //this controls the speed of the slower clock, setting to [1:0] for
simulation speed so slow clk is about every 40ns
                //for 1 Hz we have desired freq = clk freq / 2^n so 1 = 100,000,000/2^n so n =
26.57542 so I will use a 27 bit counter for about 0.74Hz

assign clk_out=COUNT[26];

always @(posedge clk) begin
    if (reset)
        COUNT = 0;
    else
        COUNT = COUNT + 1;
    end
endmodule

module rising_edge_detector(
    input clk,
    input signal,
    input reset,
    output reg outedge
);

wire slow_clk;

reg [1:0] state;
reg [1:0] next_state;

clkdiv cl(clk, reset, slow_clk);

//Combinational part
always @(*) begin
    case (state)
        2'b00 : begin
            outedge = 1'b0;
            if (~signal)
                next_state = 2'b00; //we got a low signal
            else
                next_state = 2'b01; //we got a high signal
        end
        2'b01 : begin
            outedge = 1'b1;
```

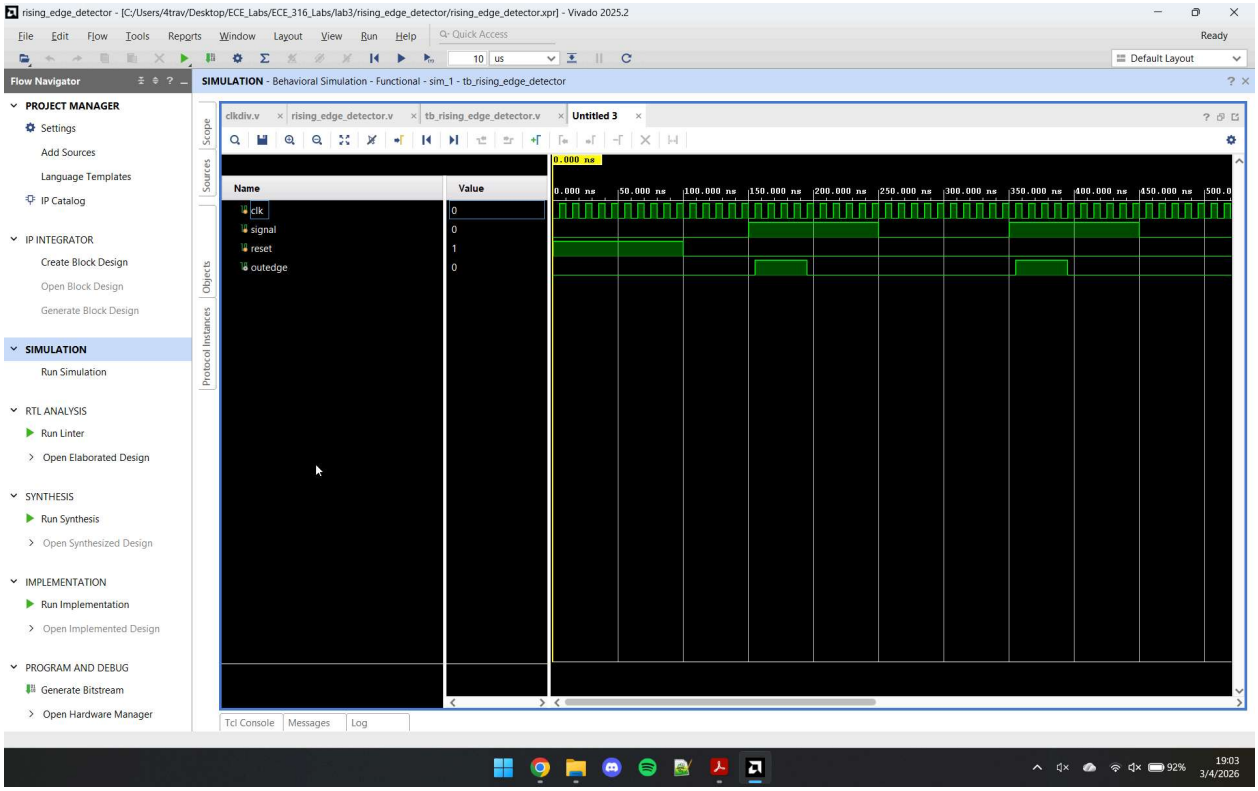
```
            if (~signal)
                next_state = 2'b00; //signal was high now low
            else
                next_state = 2'b10; //signal was high and is still high
        end
        2'b10 : begin
            outedge = 1'b0;
            if (~signal)
                next_state = 2'b00; //signal has been high and went low
            else
                next_state = 2'b10; //signal has been high and is staying high
        end
        default : begin
            next_state = 2'b00;
            outedge = 1'b0;
            end
    endcase
end

//Sequential Part
always @(posedge slow_clk or posedge reset) begin
    if(reset)
        state <= 2'b00;
    else
        state <= next_state;
end
endmodule
```

vii. Test-bench of the system

```
module tb_rising_edge_detector();
    reg clk;
    reg signal;
    reg reset;
    wire outedge;

    rising_edge_detector dut (.clk(clk), .signal(signal), .reset(reset),
        .outedge(outedge));

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        signal = 0;
```

```
        reset = 1;

        #100; //This is delay to reset everything

        reset = 0; //Set reset back to low
        #50;

        signal = 1; //Signal high -> should go to high state then wait for low state
        #100;

        signal = 0; //Signal low -> should go from wait for low to initial state
        #100;

        signal = 1; //Signal high -> should go to high state then wait for low state
        #100;

        signal = 0; //Signal low -> should go from wait for low to initial state
        #100;

    end

endmodule
```

viii. Simulation waveform



Note this is with slow clock being every 4 cycles or 40ns

ix. Constraints File (Just the uncommented portion)

```
## Clock signal - Uncomment if needed (will be used in future labs)
set_property PACKAGE_PIN W5 [get_ports clk]

        set_property IOSTANDARD LVCMOS33 [get_ports clk]
        create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports
clk]

## Switches
## Connects pin V17 (SW0 on the board) to input signal
set_property PACKAGE_PIN V17 [get_ports {signal}]
## Sets the switch to use 3.3V logic
        set_property IOSTANDARD LVCMOS33 [get_ports {signal}]

## LEDs
## Connects the output outedge in our gate module to pin U16 (LED0 on-board)
set_property PACKAGE_PIN U16 [get_ports {outedge}]
## Sets the LED to use 3.3V logic
        set_property IOSTANDARD LVCMOS33 [get_ports {outedge}]

##Buttons
set_property PACKAGE_PIN U18 [get_ports {reset}]
        set_property IOSTANDARD LVCMOS33 [get_ports {reset}]
```

Part 3

x. Completed design files (.v) of all the modules in the system

```
module clkdiv(
   input clk,
   input reset,
   output clk_out
);

reg [26:0] COUNT; //this controls the speed of the slower clock, setting to [1:0] for
simulation speed so slow clk is about every 40ns
            //for 1 Hz we have desired freq = clk freq / 2^n so 1 = 100,000,000/2^n so n =
26.57542 so I will use a 27 bit counter for about 0.74Hz

assign clk_out=COUNT[26];

always @(posedge clk) begin
   if (reset)
      COUNT = 0;
   else
```

```
        COUNT = COUNT + 1;
    end
endmodule

module time_mux_state_machine(
    input clk,
    input reset,
    input [6:0] in0,
    input [6:0] in1,
    input [6:0] in2,
    input [6:0] in3,
    output reg [3:0] an,
    output reg [6:0] sseg
);

reg [1:0] state;
reg [1:0] next_state;

always @ (*) begin
    case(state) //this is our states
        2'b00: next_state = 2'b01;
        2'b01: next_state = 2'b10;
        2'b10: next_state = 2'b11;
        2'b11: next_state = 2'b00;
        default: next_state = 2'b00; //fail safe default
    endcase
end

always @(*) begin
    case (state) //this is for assigning the sseg values
        2'b00 : sseg = in0;
        2'b01 : sseg = in1;
        2'b10 : sseg = in2;
        2'b11 : sseg = in3;
        default : sseg = 7'b1111111; //default all off
    endcase

    case (state) //this is to controll which anode is on
        2'b00 : an = 4'b1110; // right
        2'b01 : an = 4'b1101;
        2'b10 : an = 4'b1011;
        2'b11 : an = 4'b0111; //left
        default : an = 4'b1111; //default none
    endcase
```

```
end

//this is sequential part on clk
always @(posedge clk or posedge reset) begin
   if(reset)
      state <= 2'b00;
   else
      state <= next_state;
end

endmodule

module hexto7segment(
   input [3:0] x,
   output reg [6:0] r
   );

   always @(*) begin
      case (x)
         4'b0000 : r = 7'b0000001;
         4'b0001 : r = 7'b1001111;
         4'b0010 : r = 7'b0010010;
         4'b0011 : r = 7'b0000110;
         4'b0100 : r = 7'b1001100;
         4'b0101 : r = 7'b0100100;
         4'b0110 : r = 7'b0100000;
         4'b0111 : r = 7'b0001111;
         4'b1000 : r = 7'b0000000;
         4'b1001 : r = 7'b0000100;
         4'b1010 : r = 7'b0001000;
         4'b1011 : r = 7'b1100000;
         4'b1100 : r = 7'b0110001;
         4'b1101 : r = 7'b1000010;
         4'b1110 : r = 7'b0110000;
         4'b1111 : r = 7'b0111000;
         default : r = 7'b1111111; //default case added in
      endcase
   end

endmodule


module time_multiplexing_main(
   input clk,
```

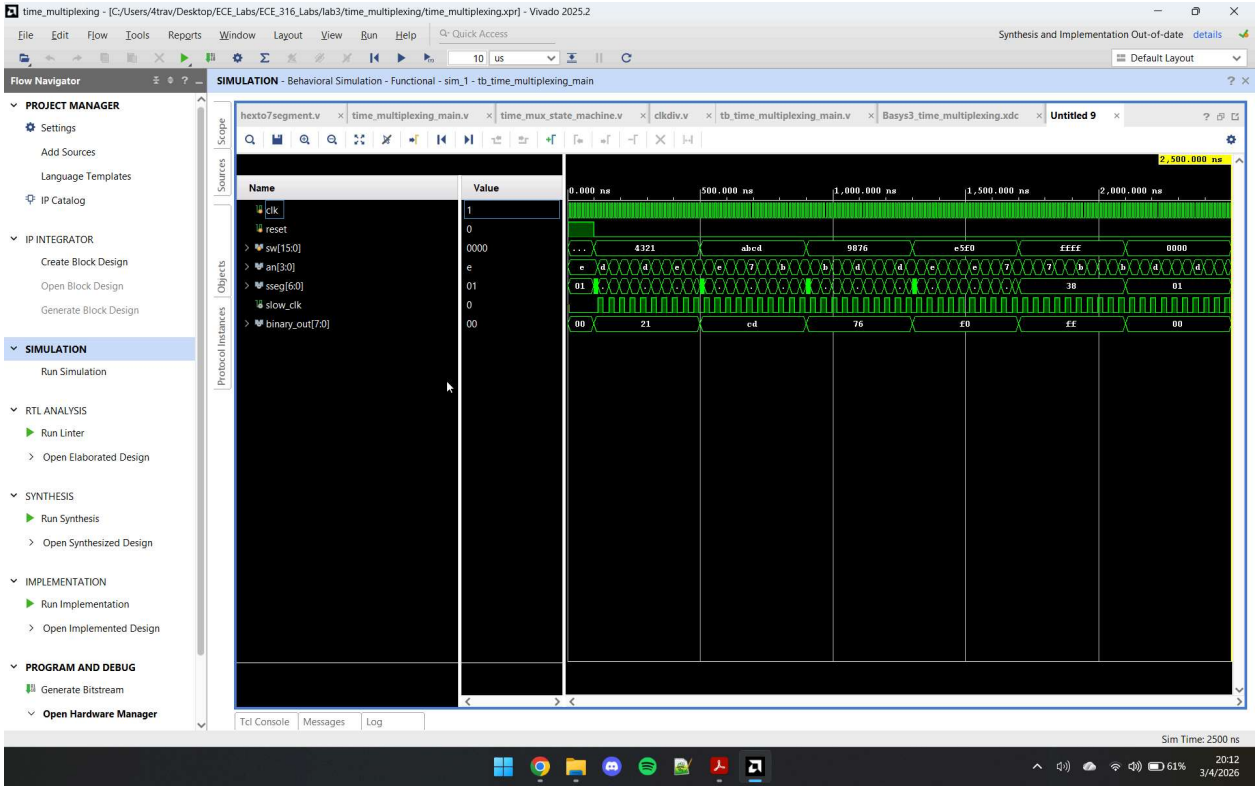
```
    input reset,
    input [15:0] sw,
    output [3:0] an,
    output [6:0] sseg,
    output slow_clk,
    output [7:0] binary_out
    );

wire [6:0] in0, in1, in2, in3;
assign binary_out = sw[7:0];

// Module instantiation of hexto7segment decoder
hexto7segment h0 (.x(sw[3:0]), .r(in0));
hexto7segment h1 (.x(sw[7:4]),   .r(in1));
hexto7segment h2 (.x(sw[11:8]),  .r(in2));
hexto7segment h3 (.x(sw[15:12]), .r(in3));

// Module instantiation of clock divider
clkdiv c0(.clk(clk), .reset(reset), .clk_out(slow_clk));

// Module instantiation of the multiplexer FSM
time_mux_state_machine f1_inst( .clk(slow_clk), .reset(reset),
      .in0(in0), .in1(in1), .in2(in2), .in3(in3),
      .an(an), .sseg(sseg));

endmodule
```

xi. Test-bench of the system

```
module tb_time_multiplexing_main();
    reg clk;
    reg reset;
    reg [15:0] sw;

    wire [3:0] an;
    wire [6:0] sseg;
    wire slow_clk;
    wire [7:0] binary_out;

    time_multiplexing_main dut (.clk(clk), .reset(reset), .sw(sw), .an(an), .sseg(sseg),
        .slow_clk(slow_clk), .binary_out(binary_out));

    always #5 clk = ~clk;

    initial begin
```

```
        clk = 0;
        reset = 1;
        sw = 16'h0000;
        #100; //forcing reset

        reset = 0; //reset reset val

        sw = 16'h4321; //test 4321
        #400; // need long delay to cycle through all since clk is now 40ns

        sw = 16'hABCD; //test ABCD
        #400;

        sw = 16'h9876; //test 9876
        #400;

        sw = 16'hE5F0; //test something random I felt like
        #400;

        sw = 16'hFFFF; //test straight Fs
        #400;

        sw = 16'h0000; //test straight 0s
        #400;

        $finish;
    end

endmodule
```

xii. Simulation waveform



xiii. Constraints File (Just the uncommented portion)

```
## Clock signal
set_property PACKAGE_PIN W5 [get_ports clk]

        set_property IOSTANDARD LVCMOS33 [get_ports clk]
        create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports
clk]

## Switches
set_property PACKAGE_PIN V17 [get_ports {sw[0]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
set_property PACKAGE_PIN V16 [get_ports {sw[1]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
set_property PACKAGE_PIN W16 [get_ports {sw[2]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
set_property PACKAGE_PIN W17 [get_ports {sw[3]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
set_property PACKAGE_PIN W15 [get_ports {sw[4]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]}]
set_property PACKAGE_PIN V15 [get_ports {sw[5]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]
set_property PACKAGE_PIN W14 [get_ports {sw[6]}]
```

```
	set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]
set_property PACKAGE_PIN W13 [get_ports {sw[7]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]
set_property PACKAGE_PIN V2 [get_ports {sw[8]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {sw[8]}]
set_property PACKAGE_PIN T3 [get_ports {sw[9]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {sw[9]}]
set_property PACKAGE_PIN T2 [get_ports {sw[10]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {sw[10]}]
set_property PACKAGE_PIN R3 [get_ports {sw[11]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {sw[11]}]
set_property PACKAGE_PIN W2 [get_ports {sw[12]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {sw[12]}]
set_property PACKAGE_PIN U1 [get_ports {sw[13]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {sw[13]}]
set_property PACKAGE_PIN T1 [get_ports {sw[14]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {sw[14]}]
set_property PACKAGE_PIN R2 [get_ports {sw[15]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {sw[15]}]

## LEDs - binary
set_property PACKAGE_PIN U16 [get_ports {binary_out[0]}]

	set_property IOSTANDARD LVCMOS33 [get_ports {binary_out[0]}]
set_property PACKAGE_PIN E19 [get_ports {binary_out[1]}]

	set_property IOSTANDARD LVCMOS33 [get_ports {binary_out[1]}]
set_property PACKAGE_PIN U19 [get_ports {binary_out[2]}]

	set_property IOSTANDARD LVCMOS33 [get_ports {binary_out[2]}]
set_property PACKAGE_PIN V19 [get_ports {binary_out[3]}]

	set_property IOSTANDARD LVCMOS33 [get_ports {binary_out[3]}]
set_property PACKAGE_PIN W18 [get_ports {binary_out[4]}]

	set_property IOSTANDARD LVCMOS33 [get_ports {binary_out[4]}]
set_property PACKAGE_PIN U15 [get_ports {binary_out[5]}]

	set_property IOSTANDARD LVCMOS33 [get_ports {binary_out[5]}]
set_property PACKAGE_PIN U14 [get_ports {binary_out[6]}]

	set_property IOSTANDARD LVCMOS33 [get_ports {binary_out[6]}]
set_property PACKAGE_PIN V14 [get_ports {binary_out[7]}]
```

```
        set_property IOSTANDARD LVCMOS33 [get_ports {binary_out[7]}]

## LED - slow clock
set_property PACKAGE_PIN L1 [get_ports {slow_clk}]
        set_property IOSTANDARD LVCMOS33 [get_ports {slow_clk}]

## 7 Seg
set_property PACKAGE_PIN W7 [get_ports {sseg[6]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {sseg[6]}]
set_property PACKAGE_PIN W6 [get_ports {sseg[5]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {sseg[5]}]
set_property PACKAGE_PIN U8 [get_ports {sseg[4]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {sseg[4]}]
set_property PACKAGE_PIN V8 [get_ports {sseg[3]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {sseg[3]}]
set_property PACKAGE_PIN U5 [get_ports {sseg[2]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {sseg[2]}]
set_property PACKAGE_PIN V5 [get_ports {sseg[1]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {sseg[1]}]
set_property PACKAGE_PIN U7 [get_ports {sseg[0]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {sseg[0]}]

## Anodes
set_property PACKAGE_PIN U2 [get_ports {an[0]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]
set_property PACKAGE_PIN U4 [get_ports {an[1]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]
set_property PACKAGE_PIN V4 [get_ports {an[2]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]
set_property PACKAGE_PIN W4 [get_ports {an[3]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]

## Buttons
set_property PACKAGE_PIN U18 [get_ports reset]
        set_property IOSTANDARD LVCMOS33 [get_ports reset]
```